

```

/**
 * server/lib/cron-scheduler.ts
 *
 * Fix #3 – 19 Cron Jobs All Miss at Midnight (Thundering Herd)
 *
 * Drop-in replacement for node-cron that staggers jobs scheduled at the
 * same time. All the midnight reset jobs (token budgets, quota counters,
 * daily strategy cycles) fire at 00:00:00 UTC simultaneously – causing
 * a massive event loop stall where every cron misses its execution.
 *
 * Usage – replace:
 *   schedule('0 0 * * *', myHandler)
 *
 * With:
 *   scheduleCron('0 0 * * *', myHandler, { label: 'reset-token-budget' })
 *
 * Jobs with the same cron expression are automatically staggered by
 * STAGGER_INTERVAL_MS between each, so they never fire simultaneously.
 */

import { schedule, ScheduledTask } from 'node-cron';
import { createLogger } from './logger.js';

const log = createLogger('cron-scheduler');

const STAGGER_INTERVAL_MS = 45_000; // 45 seconds between same-expression jobs

// Track how many jobs are registered per cron expression
const expressionCounts = new Map<string, number>();
const registeredTasks = new Map<string, ScheduledTask>();

interface CronOptions {
  label: string; // for logging
  timezone?: string;
}

/**
 * Schedules a cron job with automatic staggering.
 * Multiple jobs with the same expression are offset by 45s each.
 */
export function scheduleCron(
  expression: string,
  handler: () => void | Promise<void>,
  options: CronOptions,
): ScheduledTask {
  const count = expressionCounts.get(expression) ?? 0;
  const offset = count * STAGGER_INTERVAL_MS;

```

```

expressionCounts.set(expression, count + 1);

const wrappedHandler = () => {
  if (offset === 0) {
    // First job at this time – run immediately on cron tick
    Promise.resolve(handler()).catch(err =>
      log.error(`[Cron:${options.label}] Error:`, err)
    );
  } else {
    // Staggered jobs – delay by offset
    setTimeout(() => {
      Promise.resolve(handler()).catch(err =>
        log.error(`[Cron:${options.label}] Error:`, err)
      );
    }, offset);

    const offsetSec = Math.round(offset / 1000);
    log.debug(`[Cron:${options.label}] Scheduled with ${offsetSec}s offset to avoid thundering
herd`);
  }
};

const task = schedule(expression, wrappedHandler, {
  timezone: options.timezone ?? 'UTC',
});

registeredTasks.set(options.label, task);

const offsetSec = Math.round(offset / 1000);
log.info(
  `[Cron] Registered "${options.label}" | expr: ${expression} | ` +
  `offset: ${offsetSec}s | stagger slot: ${count}`
);

return task;
}

/**
 * Stop a named cron job.
 */
export function stopCron(label: string): void {
  const task = registeredTasks.get(label);
  if (task) {
    task.stop();
    registeredTasks.delete(label);
    log.info(`[Cron] Stopped "${label}"`);
  }
}

// — Usage example (replace your existing midnight schedules) —

```

```
//  
// import { scheduleCron } from '../lib/cron-scheduler.js';  
//  
// scheduleCron('0 0 * * *', resetContentGrinderBudget,    { label: 'reset-content-grinder' });  
// scheduleCron('0 0 * * *', resetThumbnailBudget,         { label: 'reset-thumbnail-budget' });  
// scheduleCron('0 0 * * *', resetViralOptimizerBudget,    { label: 'reset-viral-optimizer' });  
// scheduleCron('0 0 * * *', resetQuotaCounters,           { label: 'reset-quota-counters' });  
// scheduleCron('0 0 * * *', runDailyStrategyReset,        { label: 'daily-strategy-reset' });  
// scheduleCron('0 0 * * *', runComplianceCheck,           { label: 'compliance-check' });  
// scheduleCron('0 0 * * *', runMemoryConsolidation,       { label: 'memory-consolidation' });  
//  
// Result: instead of all 7 firing at 00:00:00 and all missing,  
// they fire at: 00:00:00, 00:00:45, 00:01:30, 00:02:15, 00:03:00, 00:03:45, 00:04:30
```